Public comments 2000-2025: designed sampling and LLM stance measurement

Table of contents


```
# ------------------------------------------------------------------------
# CONFIGURATION (the only block you edit)
# ------------------------------------------------------------------------
st <- list(
  seed          = 2026L,
  in_dir        = "outputs",        # Script 1's outputs/ (parquet + rds)
  out_dir       = "outputs_stance", # this script's checkpoints and results
  simulated     = TRUE,             # THE one-flag swap. FALSE at work: the
                                    # fenced SIMULATED MODE section defines
                                    # nothing and the real API path runs; no
                                    # other code branches on this.

  # --- topics and stance targets ---------------------------------------
  # Must match Script 1's tx$selected_topics. NULL in simulated mode picks the
  # top 3 topics by prevalence as stand-ins so the pipeline runs end to end.
  selected_topics = NULL,           # e.g. c(3L, 7L, 11L)
  # One proposition per selected topic, names = topic numbers as strings.
  # Stance is measured TOWARD this stated claim (favor / oppose / neutral /
  # irrelevant), so write the claim, not the topic name. NULL in simulated
  # mode auto-fills placeholders; at work this must be set or the render stops.
  propositions  = NULL,             # e.g. c("3" = "expanding local vaccination
                                    #         programs", "7" = ...)
```

```
  # --- scope ---------------------------------------------------------------
  oos_threshold = 0.20,             # PLACEHOLDER: a comment is out of scope if
                                    # its max selected-topic share falls below
                                    # this; set from the ECDF diagnostic, then
                                    # delete outputs_stance/ once and re-render

  # --- LLM API -------------------------------------------------------------
  # Raters: >= 2 models so inter-model agreement is measurable. Routing:
  # "openrouter/<slug>" -> ellmer::chat_openrouter (slug itself contains "/");
  # "ollama/<tag>"      -> ellmer::chat_ollama (local server, no data leaves);
  # anything else       -> WORK SWAP: ellmer::chat_openai(base_url =
  #                        Sys.getenv("WORK_LLM_BASE_URL")), the
  #                        OpenAI-compatible pattern for the work endpoint.
  models        = c(gemma4   = "ollama/gemma4:31b",
                    maverick = "openrouter/meta-llama/llama-4-maverick"),
  batch_calls   = 300L,             # API budget: calls per quota window
  window_hours  = 4L,               # quota window length (planning arithmetic)
  max_active    = 3L,               # concurrent requests (2-3 for local Ollama)
  rpm           = 60L,              # requests per minute throttle
  # Interactive defaults fail fast; PRODUCTION (unattended overnight run):
  # retries = 16L, retry_wait = 30L * 60L rides out full quota resets.
  retries       = 1L,
  retry_wait    = 5L,

  # --- validation ----------------------------------------------------------
  n_gold        = 60L               # size of the blind hand-coding subsample
)

suppressPackageStartupMessages({
  library(arrow)      # parquet input/output
  library(tidyverse)  # loaded LAST so dplyr verbs win
})
# Used namespaced only (install once, never attached): ellmer (>= 0.2, for
# parallel_chat_structured), irr (kappa), rlang (hash).
theme_set(theme_minimal(base_size = 12))
set.seed(st$seed)
dir.create(st$out_dir, showWarnings = FALSE)

# Checkpoint helpers (same contract as Script 1): compute once, then reload.
# Delete the file to re-fit. Caches track FILES, not code.
cache_rds <- function(path, expr) {
  if (file.exists(path)) return(readRDS(path))
  x <- force(expr); saveRDS(x, path); x
}
cache_parquet <- function(path, expr) {
  if (file.exists(path)) return(arrow::read_parquet(path))
  x <- force(expr); arrow::write_parquet(x, path); x
}
```

```
p_in  <- function(f) file.path(st$in_dir, f)
p_out <- function(f) file.path(st$out_dir, f)
```

Inputs from Script 1, validated

This script never touches the raw corpus: its frame is Script 1's paragraph table with STM theta. The gate below stops the render with a named error if Script 1 has not been run, if the theta columns are missing, or if the topic selection and propositions are inconsistent, which is cheaper than a silent failure inside the sampling code.

Adapting at work: nothing changes here. If Script 1's outputs live elsewhere, point `st$in_dir` at that folder. The check that every selected topic has a proposition is the contract between the two scripts: it is what guarantees the stance prompt always states its target.

```
required_in <- c("paragraph_theta.parquet", "topic_summary.parquet")
missing_in  <- required_in[!file.exists(p_in(required_in))]
if (length(missing_in) > 0)
  stop("Missing Script 1 output(s) in ", st$in_dir, ": ",
       str_c(missing_in, collapse = ", "),
       ". Render text_topics.qmd first.")

para_theta    <- arrow::read_parquet(p_in("paragraph_theta.parquet"))
topic_summary <- arrow::read_parquet(p_in("topic_summary.parquet"))

theta_cols <- str_subset(names(para_theta), "^topic_\\d+$")
K <- length(theta_cols)
stopifnot(
  "paragraph_theta must carry topic_1..topic_K" = K > 0,
  "paragraph_theta must carry ids, Year, tokens, text" =
    all(c("para_uid", "atom_id", "Year", "n_tokens", "text") %in%
          names(para_theta))
)

# Selected topics: at work these MUST match Script 1's tx$selected_topics.
# In simulated mode with NULL, the top 3 by prevalence stand in (PLACEHOLDER).
sel <- st$selected_topics
if (is.null(sel)) {
  if (!st$simulated)
    stop("st$selected_topics is NULL. Set it to the topics chosen in Script 1.")
     sel  <-  topic_summary  |>  dplyr::slice_max(prevalence,  n  =  3)  |>
dplyr::pull(topic)
  cat("PLACEHOLDER selected topics (simulated mode, top 3 by prevalence):",
      str_c(sel, collapse = ", "), "\n")
}
```

```
PLACEHOLDER selected topics (simulated mode, top 3 by prevalence): 1, 2, 3
```

```
stopifnot("selected topics must exist in the fitted model" = all(sel <= K))

# Propositions: the stance targets. One per selected topic, or the render stops.
props <- st$propositions
if (is.null(props)) {
  if (!st$simulated)
    stop("st$propositions is NULL. Write one stance proposition per selected ",
         "topic; stance has no meaning without a stated target.")
  props <- set_names(
    str_c("the policy direction represented by topic ", sel, " (PLACEHOLDER)"),
    as.character(sel))
  cat("PLACEHOLDER propositions auto-filled for simulated mode.\n")
}
```

```
PLACEHOLDER propositions auto-filled for simulated mode.
```

```
missing_props <- setdiff(as.character(sel), names(props))
if (length(missing_props) > 0)
  stop("No proposition for selected topic(s): ",
       str_c(missing_props, collapse = ", "))

# Verified topic names, if the analyst has produced them; numbers otherwise.
# Column named topic_label, NOT label: "label" is the stance column downstream
# and a collision there is the data-masking trap this pipeline already hit.
labels_path <- p_in("topic_labels_verified.csv")
topic_names <- if (file.exists(labels_path)) {
  readr::read_csv(labels_path, show_col_types = FALSE) |>
    dplyr::select(topic, topic_label = label)
} else {
  tibble(topic = seq_len(K), topic_label = str_c("topic ", seq_len(K)))
}
```

The frame holds 6087 paragraphs from 1200 comments, with 15 fitted topics of which 3 are selected for stance measurement.

PART 1: LLM API setup

The measurement instrument is a chat LLM used as a zero-shot annotator, so the setup enforces the defensibility practices documented in `defensible_llm_text_measurement.md`: structured output constrained to the label set (no free-text parsing), at least two models so inter-model agreement is measurable, pinned model tags at temperature 0, the proposition stated verbatim in the prompt, and a relevance gate ("irrelevant") because forcing a stance onto an off-target paragraph manufactures noise (Li & Conrad 2024).

Adapting at work: the provider swap is `st$models` only. Keys and the work base URL live in `.Renviron` (`OPENROUTER_API_KEY` now; `WORK_LLM_API_KEY` and `WORK_LLM_BASE_URL` later), never

in code. Record the exact model strings and the render date in the write-up; an unpinned model is unreproducible. The raters are blind by construction: the prompt carries the paragraph text and the proposition, never `Year`, `atom_id`, theta, or the sampling weight, so the instrument cannot infer the trend it is helping to measure.

```
# Provider routing off the model string. Wrapped in functions so pasting any
# fragment into a console keeps intermediates local (no stale globals).
make_chat <- function(model, system_prompt) {
  if (startsWith(model, "openrouter/")) {
    ellmer::chat_openrouter(
      model         = sub("^openrouter/", "", model),  # slug itself has a "/"
      system_prompt = system_prompt,
      params        = ellmer::params(temperature = 0),
      echo          = "none")
  } else if (startsWith(model, "ollama/")) {
    ellmer::chat_ollama(
      model         = sub("^ollama/", "", model),
      system_prompt = system_prompt,
      params        = ellmer::params(temperature = 0),
      echo          = "none")
  } else {
    # WORK SWAP target: OpenAI-compatible endpoint, base URL from .Renviron.
    ellmer::chat_openai(
      model         = model,
      base_url      = Sys.getenv("WORK_LLM_BASE_URL"),
      system_prompt = system_prompt,
      params        = ellmer::params(temperature = 0),
      echo          = "none")
  }
}

# Stance rater for one proposition and one model: chat + schema. Stance is
# position TOWARD the stated proposition, distinct from sentiment (Mohammad et
# al. 2016): hostile tone can still favor the target. The "irrelevant" label
# is the relevance gate for off-target paragraphs (Li & Conrad 2024).
stance_labels <- c("favor", "neutral", "oppose", "irrelevant")

stance_rater <- function(proposition, model) {
  system_prompt <- str_c(
    "You are a careful annotator measuring STANCE: the author's position ",
    "TOWARD THE STATED PROPOSITION. Stance is distinct from sentiment; an ",
    "angry or negative tone can still FAVOR the proposition, and warm praise ",
    "can OPPOSE it. Decide the position toward the target, not the mood.\n",
    "Stance target (the proposition): ", proposition, "\n",
    "First decide whether the text discusses the target at all; if it does ",
    "not, the label is irrelevant.\n",
    "Return exactly one label from: ",
    str_c(stance_labels, collapse = ", "), ".\n",
```

```
    "Judge only the text shown; ignore metadata and outside knowledge. If ",
    "the text is genuinely ambiguous, pick the closest label and set ",
    "confidence to low rather than inventing certainty.")

  schema <- ellmer::type_object(
    label      = ellmer::type_enum(stance_labels,
                   "The single best-fitting category."),
    confidence = ellmer::type_enum(c("low", "medium", "high"),
                   "How clearly the text fits the chosen label."),
    rationale  = ellmer::type_string(
                   "One short clause (<= 15 words) naming the deciding cue."))

  list(chat = make_chat(model, system_prompt), schema = schema, model = model)
}
```

```
# One structured pass over a text vector for one rater: always the same output
# columns, failed requests carried as NA labels instead of aborting the run.
classify_pass <- function(txt, ids, model_name, rater, max_active, rpm) {
  base <- tibble(id = ids, model = model_name, text = txt)
  out <- tryCatch(
    ellmer::parallel_chat_structured(
      chat       = rater$chat,
      prompts    = as.list(txt),
      type       = rater$schema,
      max_active = max_active,
      rpm        = rpm,
      on_error   = "continue"),
    error = function(e) {
      message("[", model_name, "] submission failed: ", conditionMessage(e))
      NULL
    })
  if (is.null(out))
    return(dplyr::mutate(base, label = NA_character_,
                         confidence = NA_character_,
                         rationale  = NA_character_))
  out |>
    as_tibble() |>
    dplyr::select(dplyr::any_of(c("label", "confidence", "rationale"))) |>
    dplyr::bind_cols(base) |>
    dplyr::select(id, model, text, label, confidence, rationale)
}

# Sleep-and-retry by recursion (no loops): after a pass, only the failed texts
# are re-sent, so successes never re-spend tokens. With production settings
# (retries = 16, retry_wait = 30*60) this rides out a full quota reset
# unattended; the interactive defaults fail fast so a misconfiguration
# surfaces in seconds, not hours.
```

```
classify_with_retry <- function(txt, ids, model_name, rater,
                                max_active, rpm, retries, retry_wait) {
  res    <- classify_pass(txt, ids, model_name, rater, max_active, rpm)
  failed <- dplyr::filter(res, is.na(label))
  if (nrow(failed) == 0 || retries == 0) {
    if (nrow(failed) > 0)
      warning("[", model_name, "] ", nrow(failed),
              " texts still unclassified after all retries (label = NA).")
    return(res)
  }
  message("[", model_name, "] ", nrow(failed), " of ", length(txt),
          " requests failed; sleeping ", retry_wait, "s then retrying (",
          retries, " retries left).")
  Sys.sleep(retry_wait)
  retried <- classify_with_retry(failed$text, failed$id, model_name, rater,
                                 max_active, rpm, retries - 1, retry_wait)
  res |>
    dplyr::filter(!is.na(label)) |>
    dplyr::bind_rows(retried) |>
    dplyr::arrange(match(id, ids))
}

# THE ONE SEAM between simulated and real runs. run_batch() is the only
# function the classification loop calls per batch; this definition is the
# REAL one (builds raters, calls the API). The SIMULATED MODE section below
# overrides it with a fake of identical signature. Everything downstream is
# branch-free and identical in both modes, so swapping to real data means
# flipping st$simulated (or deleting the simulated section outright) and
# nothing else.
run_batch <- function(b, proposition, models) {
  purrr::imap(models, function(model_string, model_name) {
    rater <- stance_rater(proposition, model_string)
    classify_with_retry(b$text, b$para_uid, model_name, rater,
                        st$max_active, st$rpm, st$retries, st$retry_wait)
  }) |>
    purrr::list_rbind()
}
```

PART 2: Scope, strata, and budget

Two decisions replace the sampling stage. First, **scope**: a comment is in scope only if it substantively engages at least one selected topic; everything else is excluded and the inference population is the in-scope corpus, stated as such in every write-up sentence. Second, **stratum assignment**: each in-scope comment is routed to exactly one selected topic, which fixes the proposition its paragraphs are measured against. Because the classification is a census of the in-scope paragraphs, there are no selection probabilities, no weights, and no sampling error for descriptive corpus

quantities; the uncertainty story lives entirely in measurement error, which the gold-standard validation in Part 3 quantifies.

Stratum assignment and scope

Paragraph theta is aggregated to the comment with **token weights**: for comment i with paragraphs j of x_{ij} tokens and STM shares θ_{ijk},

$$\bar{\theta}_{ik} = \frac{\sum_j x_{ij}\,\theta_{ijk}}{\sum_j x_{ij}}, \qquad h(i) = \arg\max_{k \in \mathcal{S}} \bar{\theta}_{ik},$$

where $\mathcal{S}$ is the selected-topic set and $h(i)$ the stratum. A comment whose maximum selected-topic share $\max_{k \in \mathcal{S}} \bar{\theta}_{ik}$ falls below `st$oos_threshold` is **out of scope** and excluded. The threshold is an analyst setting, not a statistical estimate; set it from the diagnostic below the way `tx$K` was set from searchK, then delete `outputs_stance/` once and re-render.

```
# Comment-level token-weighted theta aggregation, stratum assignment, and the
# out-of-scope rule, all inside one function so console-pasting stays clean.
build_frame <- function(para_theta, sel, oos_threshold) {
  para_theta |>
    dplyr::select(atom_id, Year, n_tokens,
                  dplyr::all_of(str_c("topic_", sel))) |>
    tidyr::pivot_longer(dplyr::starts_with("topic_"),
                        names_to = "topic", values_to = "theta",
                        names_prefix = "topic_",
                        names_transform = list(topic = as.integer)) |>
    dplyr::group_by(atom_id, Year, topic) |>
    dplyr::summarise(share = sum(n_tokens * theta) / sum(n_tokens),
                     .groups = "drop_last") |>
    dplyr::slice_max(share, n = 1, with_ties = FALSE) |>
    dplyr::ungroup() |>
    dplyr::rename(stratum = topic, max_share = share) |>
    dplyr::mutate(in_scope = max_share >= oos_threshold)
}
frame_all <- build_frame(para_theta, sel, st$oos_threshold)
frame     <- frame_all |> dplyr::filter(in_scope)

# The census frame: every paragraph of every in-scope comment, carrying its
# stratum (and therefore its proposition).
scope_paras <- cache_parquet(p_out("scope_paragraphs.parquet"), {
  para_theta |>
    dplyr::select(para_uid, atom_id, para_id, Year, n_tokens, text) |>
    dplyr::inner_join(frame |> dplyr::select(atom_id, stratum, max_share),
                      by = "atom_id")
})
```

```
frame_all |>
  ggplot(aes(max_share)) +
  stat_ecdf(linewidth = 1, color = viridis::viridis(1, begin = 0.3)) +
  geom_vline(xintercept = st$oos_threshold, linetype = 2, color = "grey40") +
  labs(x = "max selected-topic share per comment (token-weighted)",
       y = "cumulative share of comments",
       title = "Setting the out-of-scope threshold",
       subtitle = str_c("dashed line: current st$oos_threshold = ",
                        st$oos_threshold))
```

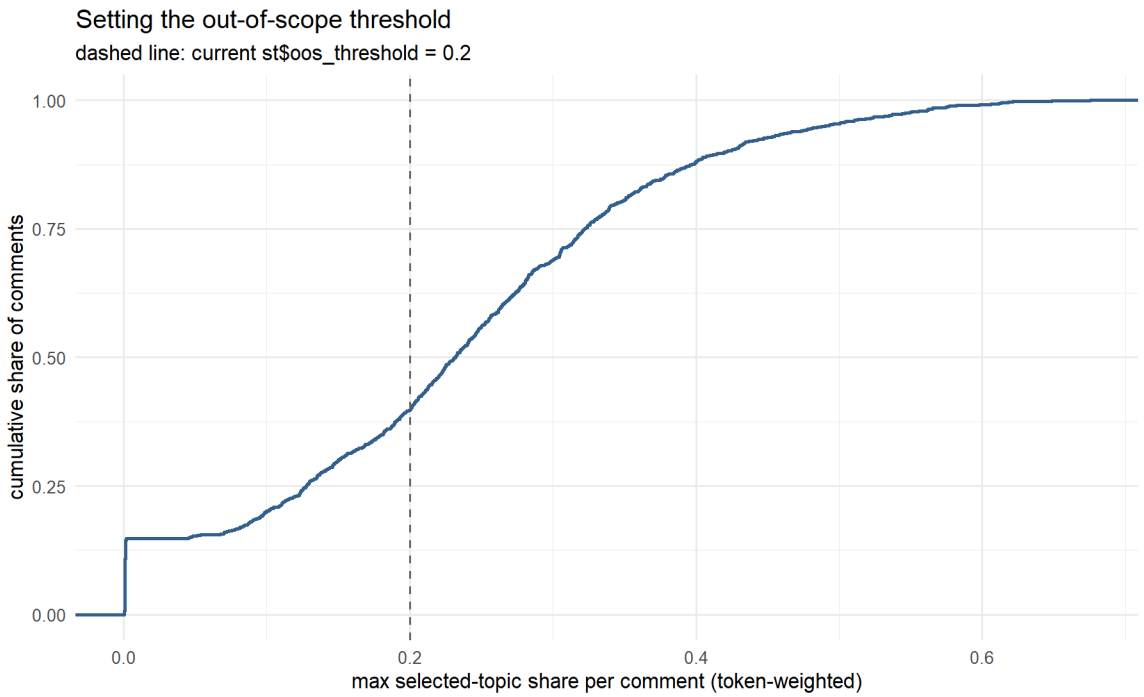

Figure 1: Out-of-scope diagnostic: ECDF of each comment's maximum selected-topic share. Read the exclusion cost of a candidate threshold directly off the curve, then set st$oos_threshold.

The rule keeps 723 of 1200 comments in scope, 60.2 percent, holding 3612 paragraphs; the excluded remainder shows negligible engagement with every selected topic.

```
scope_paras |>
  dplyr::group_by(stratum) |>
  dplyr::summarise(comments   = dplyr::n_distinct(atom_id),
                   paragraphs = dplyr::n(),
                   .groups = "drop") |>
  dplyr::left_join(topic_names, by = c("stratum" = "topic")) |>
  dplyr::relocate(stratum, topic_label) |>
  knitr::kable(caption = "Census frame by stratum: in-scope comments and the
paragraphs to classify.")
```

stratum	topic_label	comments	paragraphs
1	topic 1	267	1334
2	topic 2	229	1144
3	topic 3	227	1134

Table 1: Census frame by stratum: in-scope comments and the paragraphs to classify.

Budget arithmetic

Calls are per paragraph per model over the whole in-scope corpus, so the constraint is calendar, not validity. The batching and per-batch checkpoints in Part 3 carry the run across quota windows unattended; the arithmetic below states what that costs.

```
n_paras   <- nrow(scope_paras)
n_models  <- length(st$models)
n_calls   <- n_paras * n_models
n_windows <- ceiling(n_calls / st$batch_calls)
cat("Paragraphs to classify (census):", n_paras,
    "\nModels:", n_models,
    "\nTotal calls:", n_calls,
    "\nBudget windows of", st$batch_calls, "calls:", n_windows,
    "\nCalendar if every window is needed: about",
    n_windows * st$window_hours, "hours (",
    round(n_windows * st$window_hours / 24, 1), "days )\n")
```

```
Paragraphs to classify (census): 3612
Models: 2
Total calls: 7224
Budget windows of 300 calls: 25
Calendar if every window is needed: about 100 hours ( 4.2 days )
```

If the 4.2 days this implies do not fit the study clock, the levers are, in order of preference: fewer selected topics, a stricter `st$oos_threshold`, or one model with the second validated on the gold subsample only, each a config change with the trade-off stated in the write-up.

Inference framing for a census

A census of the in-scope corpus admits no design-based confidence interval: with every paragraph observed there is no sampling distribution, so corpus proportions are reported as exact descriptive quantities of the in-scope corpus, and the honest uncertainty statement attaches to **measurement**, the per-class error rates from the gold validation. Two disciplined consequences: no interval is put on a corpus proportion, and no claim is made beyond the in-scope corpus. If a future write-up wants inferential trend claims, treating the comments as draws from a process, that is a superpopulation framing and a design decision to raise explicitly at that point, not a default.

PART 3: Stance detection

SIMULATED MODE (flip st$simulated = FALSE at work, or delete this section)

This section is Script 2's analogue of Script 1's simulated-data section, and it is quarantined the same way: **every simulation-specific object lives inside this one section**, and the code after it carries no `st$simulated` branches. The only downstream behavior that differs between modes is data-driven, keyed on whether the planted `true_label` column exists, so nothing after this section is edited at work; the swap is `st$simulated = FALSE` in the config (this section then defines nothing), or deleting the section outright, plus purging `outputs_stance/` once so simulated checkpoints cannot reload.

What it does while active: plants a true stance per paragraph with a probability of "oppose" moving linearly in Year within each stratum (distinct slopes, so recovery is a discriminating test), fakes each configured model as a 0.85-fidelity annotator, and **overrides `run_batch()`** with an API-free version of the same signature. The census tabulation at the end then overlays the planted curves: the consensus trend should move with the planted slopes, sitting closer to the majority classes than the truth by exactly the measurement attenuation the gold metrics quantify.

```
if (st$simulated) {
  plant_truth <- function(scope_paras, sel, seed) {
    set.seed(seed)
    slopes <- set_names(seq(-0.012, 0.012, length.out = length(sel)), sel)
    scope_paras |>
      dplyr::mutate(
        p_oppose = pmin(0.85, pmax(0.05,
                     0.35 + slopes[as.character(stratum)] * (Year - 2000))),
        true_label = purrr::map_chr(p_oppose, function(p)
          sample(stance_labels, 1,
                 prob = c(0.9 * (1 - p) * 0.6,   # favor
                          0.9 * (1 - p) * 0.4,   # neutral
                          0.9 * p + 0.1 * p,     # oppose
                          0.1 * (1 - p)))))      # irrelevant
  }
  scope_paras <- plant_truth(scope_paras, sel, st$seed)

  fake_rater_run <- function(paras, model_name, seed) {
    # distinct deterministic stream per fake model
    offset <- strtoi(substr(rlang::hash(model_name), 1, 6), base = 16L)
    set.seed(seed + offset)
    paras |>
      dplyr::transmute(
        id    = para_uid,
        model = model_name,
        text  = text,
        label = purrr::map_chr(true_label, function(tl)
```

```
          if (runif(1) < 0.85) tl else sample(setdiff(stance_labels, tl), 1)),
        confidence = sample(c("low", "medium", "high"), dplyr::n(),
                            replace = TRUE, prob = c(0.15, 0.35, 0.50)),
        rationale  = "simulated response")
  }

  # Override the seam: identical signature, no API, no key, no tokens.
  run_batch <- function(b, proposition, models) {
    purrr::imap(models, function(model_string, model_name)
      fake_rater_run(b, model_name, st$seed)) |>
      purrr::list_rbind()
  }
}
```

Classification, batched to the budget and checkpointed per batch

The census reaches the API one stratum at a time, so each rater's prompt carries that stratum's proposition. Within a stratum, paragraphs are split into batches sized so one batch of all models fits one quota window (150 paragraphs times 2 models). Every batch is checkpointed to its own `.rds` the moment it completes, so an interrupted run resumes from disk: re-rendering skips finished batches and re-sends nothing.

Adapting at work: this chunk is the long one and the production retry settings belong here (`st$retries = 16`, `st$retry_wait = 30*60`). Batches that hit the quota wall mid-window sleep and retry only their failed texts. Delete a batch's checkpoint file to force re-classification of that batch alone.

```
paras_per_batch <- max(1L, st$batch_calls %/% length(st$models))

# Branch-free: the loop only knows run_batch(), whichever definition is live.
classify_stratum <- function(paras_h, stratum_id, proposition, models) {
  batches <- paras_h |>
    dplyr::mutate(batch = (dplyr::row_number() - 1L) %/% paras_per_batch + 1L)
|>
    dplyr::group_split(batch)

  purrr::map(batches, function(b) {
    ckpt <- p_out(sprintf("stance_s%02d_b%03d.rds",
                          stratum_id, dplyr::first(b$batch)))
    cache_rds(ckpt, run_batch(b, proposition, models))
  }) |>
    purrr::list_rbind()
}

stance_raw <- cache_parquet(p_out("stance_labels.parquet"), {
  scope_paras |>
    dplyr::group_split(stratum) |>
```

```
    purrr::map(function(paras_h) {
      s_id <- dplyr::first(paras_h$stratum)
      classify_stratum(paras_h, s_id, props[[as.character(s_id)]], st$models)
    }) |>
    purrr::list_rbind()
})
```

Reliability: inter-model agreement

Agreement across models is the reliability evidence, and disagreement is information, not noise: it concentrates on texts that express stance implicitly, the same texts human coders disagree on (Li & Conrad 2024). Kappa carries no validated interpretive thresholds, so it is reported alongside raw agreement and the full confusion so the reader can see *which classes* the models cannot be trusted on, per class, not on an adjective.

```
model_agreement <- function(classified) {
  wide <- classified |>
    dplyr::filter(!is.na(label)) |>
    dplyr::select(id, model, label) |>
    tidyr::pivot_wider(names_from = model, values_from = label) |>
    tidyr::drop_na()
  ratings <- as.data.frame(dplyr::select(wide, -id))
  kappa <- if (ncol(ratings) == 2) irr::kappa2(ratings)$value
           else irr::kappam.fleiss(ratings)$value
  list(kappa         = kappa,
       raw_agreement = mean(apply(ratings, 1,
                                  function(r) length(unique(r)) == 1L)),
       n_items       = nrow(ratings),
       confusion     = if (ncol(ratings) == 2)
         table(ratings[[1]], ratings[[2]], dnn = names(ratings)))
}
agree <- model_agreement(stance_raw)
cat("Inter-model agreement over", agree$n_items, "paragraphs:",
    "\n  kappa =", round(agree$kappa, 3),
    "  raw agreement =", round(agree$raw_agreement, 3), "\n")
```

```
Inter-model agreement over 1735 paragraphs:
  kappa = 0.634   raw agreement = 0.74
```

```
agree$confusion
```

```
            maverick
gemma4       favor irrelevant neutral oppose
  favor        476         34      57     46
  irrelevant    15         83      28     22
```

```
  neutral       42         19     297     46
  oppose        56         27      59    428
```

Consensus and human-LLM triage

The point estimate per paragraph is the majority vote, with ties broken alphabetically so the result is deterministic. Unanimous items are accepted from the LLMs at scale; split items are routed to a human coder, the hybrid pipeline Li and Conrad (2024) recommend. The triage file is written for the coder sorted most-disputed first.

```
mode_label <- function(x) {
  tab <- table(x)
  names(tab)[order(-as.integer(tab), names(tab))][1]
}

consensus_label <- function(classified) {
  ok <- dplyr::filter(classified, !is.na(label))
  if (nrow(ok) < nrow(classified))
    message(nrow(classified) - nrow(ok), " failed classifications dropped.")
  ok |>
    dplyr::group_by(id) |>
    dplyr::summarise(consensus = mode_label(label),
                     n_models  = dplyr::n(),
                     n_agree   = max(table(label)),
                     unanimous = max(table(label)) == dplyr::n(),
                     .groups   = "drop")
}

consensus <- consensus_label(stance_raw)

triage <- stance_raw |>
  dplyr::filter(!is.na(label)) |>
  dplyr::select(id, text, model, label) |>
  tidyr::pivot_wider(names_from = model, values_from = label) |>
  dplyr::inner_join(consensus, by = "id") |>
  dplyr::mutate(flag = !unanimous) |>
  dplyr::arrange(dplyr::desc(flag), n_agree)
readr::write_csv(triage |> dplyr::filter(flag), p_out("triage_for_review.csv"))
```

Of 1735 classified paragraphs, 451 (26.0 percent) are non-unanimous and written to `triage_for_review.csv` for human coding; unanimous items are accepted from the ensemble.

Validity: gold-standard subsample

Reliability without validity is agreement on the wrong answer, so a blind subsample is drawn for hand-coding: a stratified simple random sample, the one probability draw left in the pipeline, because the per-class error rates it yields only generalize to the census if its selection probabilities are known. Equal counts per stratum, text and id only, no model labels attached, so the coder

cannot anchor on the machine. Per-class precision, recall, and F1 plus macro-F1 and kappa against the human labels are the defensibility record; macro-F1, not accuracy, is the headline because the classes are imbalanced. In simulated mode the planted truth stands in for the human coder, which closes the loop: the metrics should read as a noisy-but-decent annotator by construction (0.85 fidelity), and if they do not, the plumbing is wrong.

```
set.seed(st$seed)   # the gold draw must be reproducible across renders
n_gold_h <- ceiling(st$n_gold / length(sel))   # target per stratum
gold_ids <- scope_paras |>
  dplyr::group_by(stratum) |>
  dplyr::mutate(.draw = sample(dplyr::n())) |>  # random rank within stratum
  dplyr::filter(.draw <= n_gold_h) |>           # keeps whole stratum if small
  dplyr::ungroup() |>
  dplyr::pull(para_uid)

scope_paras |>
  dplyr::filter(para_uid %in% gold_ids) |>
  dplyr::select(id = para_uid, text) |>
  readr::write_csv(p_out("gold_to_code.csv"))

# Data-driven, not flag-driven: planted truth stands in for the human coder
# exactly when it exists; otherwise the hand-coded file is used when present.
gold <- if ("true_label" %in% names(scope_paras)) {
  scope_paras |>
    dplyr::filter(para_uid %in% gold_ids) |>
    dplyr::select(id = para_uid, gold_label = true_label)
} else if (file.exists(p_out("gold_coded.csv"))) {
  readr::read_csv(p_out("gold_coded.csv"), show_col_types = FALSE)
} else NULL

validate_against_gold <- function(classified, gold, model_name) {
  joined <- classified |>
    dplyr::filter(model == model_name, !is.na(label)) |>
    dplyr::inner_join(gold, by = "id")
  classes <- sort(union(joined$label, joined$gold_label))
  per_class <- purrr::map_dfr(classes, function(cl) {
    tp <- sum(joined$label == cl & joined$gold_label == cl)
    fp <- sum(joined$label == cl & joined$gold_label != cl)
    fn <- sum(joined$label != cl & joined$gold_label == cl)
    precision <- if (tp + fp > 0) tp / (tp + fp) else NA_real_
    recall    <- if (tp + fn > 0) tp / (tp + fn) else NA_real_
    f1 <- if (!is.na(precision) && !is.na(recall) && precision + recall > 0)
            2 * precision * recall / (precision + recall) else NA_real_
    tibble(class = cl, precision = precision, recall = recall, f1 = f1,
           support = sum(joined$gold_label == cl))
  })
  list(per_class = per_class,
       macro_f1  = mean(per_class$f1, na.rm = TRUE),
```

```
       accuracy  = mean(joined$label == joined$gold_label),
       kappa     = irr::kappa2(dplyr::select(joined, gold_label, label))$value,
       confusion = table(gold = joined$gold_label, pred = joined$label))
}

if (!is.null(gold)) {
  validity <- purrr::imap(st$models, function(model_string, model_name)
    validate_against_gold(stance_raw, gold, model_name))
  purrr::iwalk(validity, function(v, nm) {
    cat("\n[", nm, "] macro-F1 =", round(v$macro_f1, 3),
        " accuracy =", round(v$accuracy, 3),
        " kappa vs gold =", round(v$kappa, 3), "\n")
    print(v$per_class |> dplyr::mutate(dplyr::across(
      c(precision, recall, f1), ~ round(.x, 3))))
  })
} else {
  cat("No gold labels yet: hand-code gold_to_code.csv, save it as",
      "gold_coded.csv with columns id, gold_label, and re-render.",
      "Until then the stance labels are unvalidated.\n")
}
```

```
[ gemma4 ] macro-F1 = 0.437  accuracy = 0.417  kappa vs gold = 0.149
# A tibble: 4 × 5
  class      precision recall     f1 support
  <chr>          <dbl>  <dbl>  <dbl>   <int>
1 favor          0.444  0.364  0.4        11
2 irrelevant     0      0     NA           2
3 neutral        0.444  0.444  0.444       9
4 oppose         0.438  0.5    0.467      14

[ maverick ] macro-F1 = 0.413  accuracy = 0.389  kappa vs gold = 0.111
# A tibble: 4 × 5
  class      precision recall     f1 support
  <chr>          <dbl>  <dbl>  <dbl>   <int>
1 favor          0.4    0.364  0.381      11
2 irrelevant     0      0     NA           2
3 neutral        0.444  0.444  0.444       9
4 oppose         0.4    0.429  0.414      14
```

Corpus proportions of stance by topic and Year

Estimation is a tabulation: the consensus label over the census of in-scope paragraphs, by stratum and by stratum and Year. These are exact descriptive quantities of the in-scope corpus, so no interval is attached to them; the uncertainty that matters is measurement error, quantified per class by the gold metrics above, and any write-up sentence should pair a proportion with the relevant class's validated error rate. Misclassification attenuates proportions toward the majority classes,

so levels are read jointly with the gold confusion, while trends are more robust when the error rates are stable over the corpus.

In simulated mode the plot overlays the planted truth (dashed): the gap between the solid consensus line and the dashed truth is exactly that measurement attenuation, by construction, since consensus over two 0.85-fidelity annotators must sit closer to the majority classes than the truth does. The pipeline is validated by the gold metrics reading as a noisy-but-decent annotator and by the consensus trend moving with the planted slopes, not by level coincidence.

```
est_frame <- scope_paras |>
  dplyr::inner_join(consensus, by = c("para_uid" = "id")) |>
  dplyr::left_join(topic_names, by = c("stratum" = "topic"))

stance_by_topic <- est_frame |>
  dplyr::count(topic_label, consensus) |>
  dplyr::group_by(topic_label) |>
  dplyr::mutate(p = n / sum(n)) |>
  dplyr::ungroup()

stance_by_year <- est_frame |>
  dplyr::group_by(topic_label, Year) |>
  dplyr::summarise(n_paras  = dplyr::n(),
                   p_oppose = mean(consensus == "oppose"),
                   .groups  = "drop")

stance_by_topic |>
  dplyr::mutate(p = round(p, 3)) |>
  tidyr::pivot_wider(id_cols = topic_label, names_from = consensus,
                     values_from = p, values_fill = 0) |>
  knitr::kable(caption = "Stance distribution by topic stratum (consensus label)
over the census of in-scope paragraphs. Exact corpus proportions; read levels
jointly with the per-class gold error rates.")
```

topic_label	favor	irrelevant	neutral	oppose
topic 1	0.495	0.110	0.263	0.132
topic 2	0.414	0.104	0.253	0.229
topic 3	0.346	0.095	0.178	0.381

Table 2: Stance distribution by topic stratum (consensus label) over the census of in-scope paragraphs. Exact corpus proportions; read levels jointly with the per-class gold error rates.

```
p_year <- stance_by_year |>
  ggplot(aes(Year, p_oppose, color = topic_label)) +
  geom_line(linewidth = 1) +
  geom_point(aes(size = n_paras), alpha = 0.6) +
```

```
  scale_color_viridis_d(end = 0.85, name = "stratum") +
  scale_size_continuous(name = "paragraphs", range = c(0.5, 3)) +
  labs(x = NULL, y = "P(oppose)",
       title = "Stance toward the stratum proposition over time")

if ("true_label" %in% names(scope_paras)) {   # planted truth present
  truth_line <- scope_paras |>
    dplyr::left_join(topic_names, by = c("stratum" = "topic")) |>
    dplyr::distinct(topic_label, Year, p_oppose)
  p_year <- p_year +
    geom_line(data = truth_line, aes(Year, p_oppose, color = topic_label),
              linetype = 2, linewidth = 0.6, show.legend = FALSE)
}
print(p_year)
```

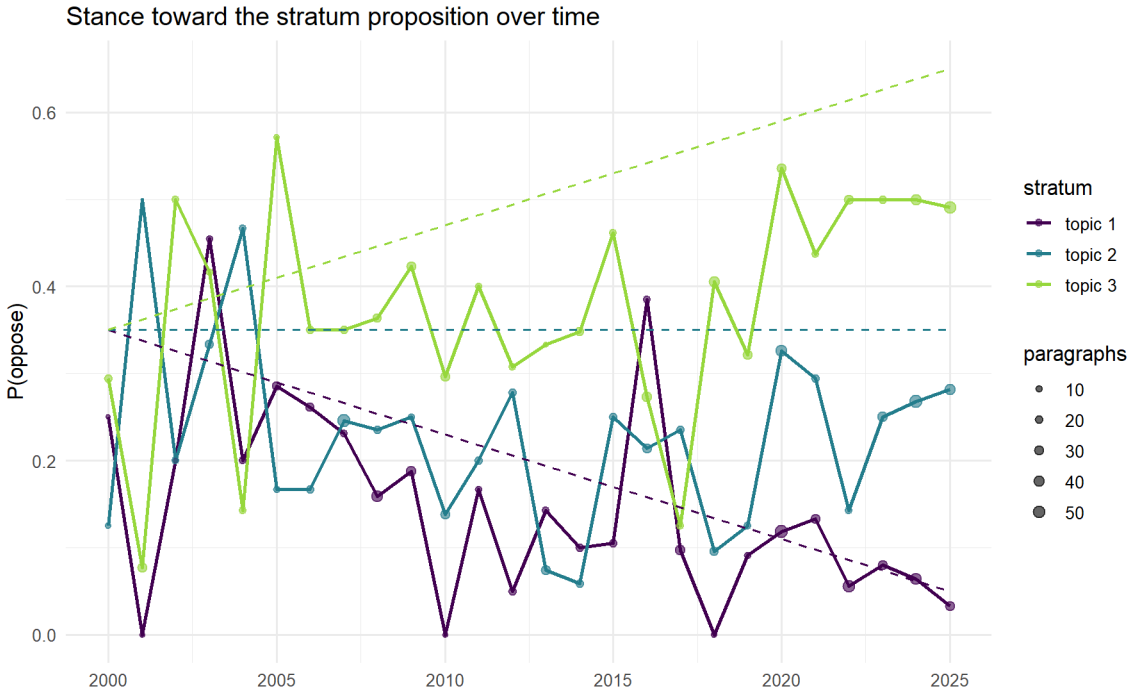

Figure 2: Share of in-scope paragraphs opposing the stratum proposition, by topic and Year (census tabulation of the consensus label). Simulated mode overlays the planted truth (dashed); the vertical gap is measurement attenuation, quantified by the gold metrics.

Handoff and record of the measurement

```
arrow::write_parquet(stance_by_topic, p_out("stance_by_topic.parquet"))
arrow::write_parquet(stance_by_year,  p_out("stance_by_year.parquet"))

cat("outputs_stance/ written:\n",
```

```
    "- scope_paragraphs.parquet   the in-scope census frame the LLM saw\n",
    "- stance_s*_b*.rds           per-batch checkpoints (resume on rerun)\n",
    "- stance_labels.parquet      one row per paragraph x model\n",
    "- triage_for_review.csv      non-unanimous items for the human coder\n",
    "- gold_to_code.csv           blind validation subsample\n",
    "- stance_by_topic.parquet, stance_by_year.parquet  corpus proportions\n",
    "\nMeasurement record: models", str_c(st$models, collapse = " | "),
    "; temperature 0; run date", format(Sys.Date()), "\n")
```

```
outputs_stance/ written:
 - scope_paragraphs.parquet   the in-scope census frame the LLM saw
 - stance_s*_b*.rds           per-batch checkpoints (resume on rerun)
 - stance_labels.parquet      one row per paragraph x model
 - triage_for_review.csv      non-unanimous items for the human coder
 - gold_to_code.csv           blind validation subsample
 - stance_by_topic.parquet, stance_by_year.parquet  corpus proportions

Measurement record: models ollama/gemma4:31b | openrouter/meta-llama/llama-4-
maverick ; temperature 0; run date 2026-07-09
```

The defensibility record for the write-up: the construct is stance toward a stated proposition per stratum (targets in `st$propositions`); the measurement covers a census of the in-scope corpus, with scope defined by the documented threshold rule; reliability is the inter-model kappa, raw agreement, and confusion above; validity is the per-class metrics against the hand-coded gold, drawn as a stratified probability subsample; non-unanimous items went to human review; and the model tags, temperature, prompts, and run date are pinned in this document.